

DECODE YOUR COURSE

Frequently Asked Technical Interview Questions

1. Data Structures & Algorithms

Array vs. Linked List

- Array: Contiguous memory allocation, fixed size, fast access ($O(1)$ for index-based retrieval).
- Linked List: Non-contiguous memory, dynamic size, slow access ($O(n)$ traversal), but insertion and deletion are efficient ($O(1)$ for head/tail).

Types of Linked Lists

- Singly Linked List: Each node has a pointer to the next node.
- Doubly Linked List: Each node has pointers to both the previous and next node.
- Circular Linked List: The last node points back to the first node.

Time Complexity of BST Search

- Best/Average Case: $O(\log n)$ (Balanced BST).
- Worst Case: $O(n)$ (Unbalanced BST, behaves like a linked list).

Hash Table & Collision Handling

- Hash table: Uses a hash function to map keys to indices for quick access.
- Collision Handling: Separate chaining (linked list at each index), open addressing (linear probing, quadratic probing, double hashing).

Stack vs. Queue

- Stack: LIFO (Last In First Out) (e.g., undo feature).
- Queue: FIFO (First In First Out) (e.g., printer queue).

BFS vs. DFS

- BFS (Breadth-First Search): Uses a queue, explores level by level.
- DFS (Depth-First Search): Uses a stack (or recursion), explores deeply before backtracking.

Cycle Detection in Linked List

- Floyd's Cycle Detection Algorithm (Tortoise and Hare): Use two pointers (fast and slow) to detect cycles.

Quick Sort & Worst-Case Complexity

- Worst Case: $O(n^2)$ (when pivot is always the smallest/largest element).
- Average Case: $O(n \log n)$.

Sorting Algorithms & Fastest One

- Merge Sort and Quick Sort are generally the fastest for large datasets ($O(n \log n)$).
- Bubble Sort and Insertion Sort are slow ($O(n^2)$).

Dynamic Programming Example

- Fibonacci Series (Optimized using DP)

```
def fib(n, memo={}):  
    if n in memo: return memo[n]  
    if n <= 2: return 1  
    memo[n] = fib(n-1, memo) + fib(n-2, memo)  
    return memo[n]
```

2. Object-Oriented Programming (OOP)

Four Pillars of OOP

1. Encapsulation (Data hiding)
2. Abstraction (Hiding implementation details)
3. Inheritance (Code reusability)
4. Polymorphism (Multiple behaviors for the same function)

Abstraction vs. Encapsulation

- Abstraction: Hiding implementation details, exposing only functionality.
- Encapsulation: Wrapping data and methods together in a class.

Overloading vs. Overriding

- Overloading: Same method name, different parameters (compile-time polymorphism).
- Overriding: Same method name in parent and child class, modified in child class (runtime polymorphism).

Constructor vs. Destructor

- Constructor: Initializes an object when created.
- Destructor: Cleans up memory when an object is destroyed.

Interface vs. Abstract Class

- Interface: Only method declarations, no implementation.
- Abstract Class: Can have both declared and defined methods.

3. Operating Systems

Process vs. Thread

- Process: Independent execution unit, separate memory.
- Thread: Lightweight, shares memory with the process.

Deadlock & Prevention

- Deadlock: When processes wait for resources indefinitely.
- Prevention: Avoid circular wait, allocate resources safely.

Virtual Memory

- Uses disk storage to simulate additional RAM when needed.

Paging vs. Segmentation

- Paging: Fixed-size memory blocks.
- Segmentation: Variable-size memory blocks.

Race Condition & Avoidance

- Race Condition: Multiple threads modifying shared data simultaneously.
- Solution: Use mutex locks or semaphores.

CPU Scheduling Algorithms

- FCFS (First Come First Serve)
- SJF (Shortest Job First)
- Round Robin (Time-Slicing)

4. Cyber Security

Encryption

- Converts data into unreadable format.

SQL Injection Prevention

- Use prepared statements instead of raw queries.

XSS vs. CSRF

- XSS: Injecting scripts into a site.
- CSRF: Unauthorized actions by authenticated users.

5. Software Testing

Software Testing Fundamentals

- What is Software Testing? Ensuring software functions correctly by identifying defects.
- Types of Testing: Manual, Automated, Functional, Non-functional.

SDLC and Testing

- Role of Testing in SDLC: Ensures quality at every stage.
- Testing in Agile vs. Waterfall: Agile has continuous testing; Waterfall has a dedicated testing phase.

Testing Levels

- Unit Testing: Verifies individual components (e.g., functions, methods).
- Integration Testing: Ensures different modules work together.
- System Testing: Evaluates the entire system's behavior.

Test Case Design

- Black Box Testing: Tests software functionality without knowing internal code.
- White Box Testing: Tests internal logic and code structure.
- Test Case Components: Test scenario, steps, expected result, actual result.

Defect Management

- Bug Life Cycle: New → Assigned → In Progress → Fixed → Retested → Closed.
- Severity vs. Priority: Severity is impact level; Priority is fixing urgency.

6. DBMS

Normalization & Normal Forms

- 1NF: No repeating columns.
- 2NF: 1NF + No partial dependencies.
- 3NF: 2NF + No transitive dependencies.

SQL vs. NoSQL

- SQL: Structured, relational databases (MySQL, PostgreSQL).
- NoSQL: Flexible schema, document-based (MongoDB, Firebase).

ACID Properties

1. Atomicity: Complete or no transaction.
2. Consistency: Valid data state.
3. Isolation: Transactions don't interfere.
4. Durability: Data is permanent after commit.

Joins in SQL

- INNER JOIN: Matching rows.
- LEFT JOIN: All from left, matching from right.
- RIGHT JOIN: All from right, matching from left.

DELETE vs. TRUNCATE vs. DROP

- DELETE: Removes rows (can use WHERE).
- TRUNCATE: Removes all rows, no rollback.
- DROP: Deletes table structure.

7. Computer Networks

TCP vs. UDP

- TCP: Reliable, connection-oriented (e.g., HTTP, FTP).
- UDP: Faster, connectionless (e.g., DNS, VoIP).

OSI Model Layers

1. Physical
2. Data Link
3. Network
4. Transport
5. Session
6. Presentation
7. Application

HTTP vs. HTTPS

- HTTPS: Encrypted communication using SSL/TLS.

Firewall

- Security system that blocks unauthorized access to networks.

Coding Questions to Practice

Easy

1. Print "Hello, World!"
2. Find the maximum of three numbers.
3. Check if a number is even or odd.
4. Reverse a string.
5. Find the factorial of a number (recursive & iterative).
6. Check if a string is a palindrome.
7. Find the sum of digits of a number.
8. Find the largest element in an array.
9. Find the second largest element in an array.
10. Check if an element exists in an array.
11. Merge two sorted arrays.
12. Count the number of vowels in a string.
13. Find the length of a string without using len().
14. Remove duplicates from an array.
15. Swap two numbers without using a third variable.
16. Check if a number is prime.
17. Find the GCD (Greatest Common Divisor) of two numbers.
18. Check if a given year is a leap year.
19. Reverse an array in-place.
20. Find the missing number in an array of 1 to N.

Medium

1. Find all prime numbers up to N (Sieve of Eratosthenes).
2. Check if two strings are anagrams.
3. Find the first non-repeating character in a string.
4. Check if a number is an Armstrong number.
5. Find the intersection of two arrays.
6. Rotate an array by K positions.
7. Find the common elements in three sorted arrays.
8. Implement Binary Search.
9. Find the square root of a number without using `sqrt()`.
10. Find the subarray with the maximum sum (Kadane's Algorithm).
11. Sort an array using Merge Sort.
12. Sort an array using Quick Sort.
13. Find all permutations of a string.
14. Find the longest common prefix among strings in an array.
15. Find the first occurrence of a number in a sorted array (Binary Search).
16. Find the majority element in an array (appears more than $N/2$ times).
17. Reverse words in a string.
18. Check if a linked list has a cycle (Floyd's Cycle Detection).
19. Find the middle node of a linked list.
20. Find the kth largest element in an array.

Hard [LEET CODE]

1. Median of Two Sorted Arrays

- Problem Number: 4
- Description: Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays. The overall run time complexity should be $O(\log(m+n))$.

2. Regular Expression Matching

- Problem Number: 10
- Description: Implement regular expression matching with support for `'.'` and `'*'` where:
 - `'.'` Matches any single character.
 - `'*'` Matches zero or more of the preceding element. The matching should cover the entire input string (not partial).

3. Trapping Rain Water

- Problem Number: 42
- Description: Given `n` non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.
[datasciencebyexample.com](https://leetcode.com/problems/trapping-rain-water/)

4. N-Queens

- Problem Number: 51
- Description: The `n`-queens puzzle is the problem of placing `n` queens on an $n \times n$ chessboard such that no two queens attack each other. Given an integer `n`, return all distinct solutions to the `n`-queens puzzle.

5. Sudoku Solver

- Problem Number: 37
- Description: Write a program to solve a Sudoku puzzle by filling the empty cells. A solution must satisfy all Sudoku rules, and the input puzzle will have a unique solution.

6. Word Ladder II

- Problem Number: 126
- Description: Given two words, begin Word and end Word, and a dictionary's word list, find all the shortest transformation sequences from begin Word to end Word, such that:
 - Only one letter can be changed at a time.
 - Each transformed word must exist in the word list. Return all such possible transformation sequences.

7. Largest Rectangle in Histogram

- Problem Number: 84
- Description: Given an array of integers representing the histogram's bar height where the width of each bar is 1, find the area of the largest rectangle in the histogram.

8. Binary Tree Maximum Path Sum

- Problem Number: 124
- Description: Given a non-empty binary tree, find the maximum path sum. For this problem, a path is defined as any sequence of nodes from some starting node to any node in the tree along the parent-child connections. The path must contain at least one node and does not need to go through the root.

9. LFU Cache

- Problem Number: 460
- Description: Design and implement a data structure for a Least Frequently Used (LFU) cache. Implement the LFU Cache class with methods to get and put items, ensuring that the cache operates in $O(1)$ time complexity.

10. Palindrome Partitioning II

- Problem Number: 132
- Description: Given a string s , partition s such that every substring of the partition is a palindrome. Return the minimum cuts needed for a palindrome partitioning of s .

Useful You Tube Videos

1. Data Structures and Algorithms

<https://youtu.be/8hly31xKli0?si=yt-ZbVeTswvmO2DV>

2. Object Oriented Programming

https://youtu.be/SiBw7os-zI?si=JG88Os4hY8kTj_Bb

3. Operating Systems

<https://youtu.be/yK1uBHPdp30?si=9ktspmPzAKHQeZbP>

4. Cyber Security

<https://youtu.be/hXSFdwIOfnE?si=MLOdS-Tw40r114ol>

5. Software Testing

<https://www.youtube.com/live/sO8eGL6SFsA?si=T0ggfdwGNfe17rjm>

6. Data Base Management System

https://youtu.be/c5HAwKX-suM?si=dFcSUaR4IU4_V32g

7. Computer Networks

<https://youtu.be/qiQR5rTSshw?si=VbvquELNmzJmrdH2>